Programming SQL Server 2016

Table JOIN

Topics

- Table JOIN explained
- Types of JOIN
- INNER JOIN
- OUTER JOINS (LEFT, RIGHT)
- FULL OUTER JOIN
- CROSS JOIN

.

Homework

- All the queries are based Northwind sample database

1. Show all orders from Orders for each customer along with CustomerID, ComapnyName, OrderID, OrderDate.
2. Show all orders from Orders for each customer along with CusomerID, ComapnyName, OrderID, OrderDate and also you want to include all the rows from Customers table. [Hint: left outer join]
3. You have to prepare a list of cities in which there are customers along with any employees in these cities. In addition, if there is an employee in a city with no customers, they should be included. Include CustomerID, City from Customers table and FirstName, City from employees table.

Important points

-
-
-

Summary

-
-
-

Table JOIN

- A normalized database is one where the data has been broken out from larger tables into many smaller tables for the purpose of eliminating repeating data, saving space, improving performance, and increasing data integrity
- When we are operating in a normalized environment, we frequently run into situations in which not all of the information that we want is in one table - they are spread across many tables.
- When we want to retrieve data from more than one table JOINs come into play
- A join puts the information from two tables together into one result set.

JOIN Types

- We can combine data from tables using various types of JOIN.
- JOINs are mainly three types – INNER, OUTER and CROSS
- OUTER joins are of three type – LEFT, RIGHT and FULL
- We summarize as
  - → INNER JOIN
  - → LEFT OUTER JOIN
  - → RIGHT OUTER JOIN
  - → FULL OUTER JOIN
  - → CROSS JOIN

Start learning table JOIN

- Before Proceeding further lets create two tables and populate with some sample data
- Tables are shown below

products			
	Column name	**Data Type**	**Nullable**
PK	productid	INT	No
	productname	VARCHAR(30)	No
	unitprice	MONEY	No

sales summary			
	Colum name	**Data type**	**Nullable**
PK	id	INT	No
	[date]	DATE	No
	quantity	INT	No
FK	productid	INT	No

- Data in products table

productid	productname	unitprice
1	Tab XN50	12000.00
2	Desire 816	28000.00
3	Lenovo Yoga 2	18000.00
4	Nexus 7	22000.00

- Data in sale summary table

id	date	quantity	productid
1	2014-10-22	6	1
2	2014-10-22	3	2
3	2014-10-22	2	4
4	2014-10-23	5	1
5	2014-10-23	2	2
6	2014-10-23	2	4
7	2014-10-24	5	1
8	2014-10-24	2	2
9	2014-10-24	2	4

- Let's build the table populate data

Try this

```
USE ESADR22
GO
CREATE TABLE products
(
    productid INT PRIMARY KEY,
    productname VARCHAR(30) NOT NULL,
    unitprice MONEY NOT NULL
)
GO
CREATE TABLE [sales summary]
(
    id INT PRIMARY KEY,
    [date] DATE NOT NULL,
    quantity INT NOT NULL,
    productid INT NOT NULL
    REFERENCES products (productid)
)
GO
INSERT INTO products
VALUES (1, 'Tab XN50', 12000.00),
(2, 'Desire 816', 28000.00),
(3, 'Lenovo Yoga 2', 18000.00),
(4, 'Nexus 7', 22000.00)
GO
SELECT * FROM products
GO
INSERT INTO [sales summary]
VALUES (1, '2014-10-22', 6, 1),
(2, '2014-10-22', 3, 2),
(3, '2014-10-22', 2, 4),
(4, '2014-10-23', 5, 1),
(5, '2014-10-23', 2, 2),
(6, '2014-10-23', 2, 4),
(7, '2014-10-24', 5, 1),
(8, '2014-10-24', 5, 2),
(9, '2014-10-24', 2, 4)
GO
SELECT * FROM [sales summary]
GO
```

Using JOIN

- By using joins, you can retrieve data from two or more tables based on logical relationships between the tables.
- Joins indicate how Microsoft SQL Server should use data from one table to select the rows in another table.
- A join condition defines the way two tables are related in a query by:
  - → Specifying the column from each table to be used for the join. A typical join condition specifies a foreign key from one table and its associated key in the other table.
  - → Specifying a logical operator (for example, = or <>,) to be used in comparing values from the columns.

JOIN syntax

```
SELECT column-list
FROM first-table
join-type second-table
ON join-condition
```

- join-type - specifies what kind of join is performed: an inner, outer, or cross join.
- join-condition - defines the predicate to be evaluated for each pair of joined rows.
- first-table - the table on the left of join type is often called left table
- second-table - the table on the right of join type is often called right table

INNER JOIN

- Inner join is the most typical join operation, which uses some comparison operator like = or <>

- An inner join matches rows between the two tables specified in the INNER JOIN statement based on the values in common columns from each table.
- Based on the table we created, suppose we want to retrieve all products along with sales data.
- Since information is in two table, we must use JOIN
- Inner join can be useful here, matching should be productid in both tables

Try this

```
SELECT productname, [date], quantity
FROM products p
INNER JOIN [sales summary] s
ON p.productid = s.productid
GO
```

→ INNER JOIN is default join type so you can omit INNER keyword

- Output

productname	date	quantity
Desire 816	2014-10-22	3
Desire 816	2014-10-23	2
Desire 816	2014-10-24	5
Nexus 7	2014-10-22	2
Nexus 7	2014-10-23	2
Nexus 7	2014-10-24	2
Tab XN50	2014-10-22	6
Tab XN50	2014-10-23	5
Tab XN50	2014-10-24	5

→ Look one product (Lenovo Yoga 2) is not included

→ INNER JOIN only includes records from the first table that has matching records in second table

→ Since product Lenovo Yoga 2 (productid 4) has no entry in sales summary, it is not included

- When multiple tables are referenced in a single query, all column references must be unambiguous.
- In the previous example both products and sales summary have a column named productid, so it must be referenced explicity with table reference like below

```
SELECT p.productid, productname, [date], quantity
FROM products p
INNER JOIN [sales summary] s
ON p.productid = s.productid
ORDER BY productname
GO
```

- Let's try more
- Suppose we want to retrieve all product along with total quantity sold of each

Try this

```
SELECT productname, SUM(quantity) 'quantity sold'
FROM products p
INNER JOIN [sales summary] s
ON p.productid = s.productid
GROUP BY productname
GO
```

- Result

productname	quantity sold
Desire 816	10
Nexus 7	6
Tab XN50	16

OUTER JOIN

- Inner joins return rows only when there is at least one row from both tables that matches the join condition. Inner joins eliminate the rows that do not match with a row from the other table.
- Outer joins, however, return all rows from at least one of the tables or views mentioned in the FROM clause.
- The LEFT JOIN keyword returns all rows from the left table, with the matching rows in the right table (table2). The result is NULL in the right side when there is no match.
- Suppose, we want to retrieve all products along with sales data and we want to include all products even if those have no sales record.
- We can do it using LEFT OUTER JOIN

Try this

```
SELECT productname, [date], quantity
FROM products p
LEFT OUTER JOIN [sales summary] s
ON p.productid = s.productid
ORDER BY productname
GO
```

→ OUTER keyword is optional in SQL Server, you can write LEFT JOIN only

- Result

productname	date	quantity
Desire 816	2014-10-22	3
Desire 816	2014-10-23	2
Desire 816	2014-10-24	5
Lenovo Yoga 2	NULL	NULL
Nexus 7	2014-10-22	2
Nexus 7	2014-10-23	2
Nexus 7	2014-10-24	2
Tab XN50	2014-10-22	6
Tab XN50	2014-10-23	5
Tab XN50	2014-10-24	5

- The RIGHT JOIN keyword returns all rows from the right table with the matching rows in the left table. The result is NULL in the left side when there is no match.

Try this

```
SELECT productname, [date], quantity
FROM [sales summary] s
RIGHT JOIN products p
ON p.productid = s.productid
ORDER BY productname
GO
```

- The result is the same as the previous example

FULL OUTER JOIN

- FULL OUTER JOIN includes all rows from both tables, regardless of whether or not the other table has a matching value.
- To retain the nonmatching information with matching information use FULL OUTER JOIN.
- Let's create two tables and insert some sample data to test full outer join.

Try this

```
CREATE TABLE tsps
(
    id INT PRIMARY KEY,
    [name] VARCHAR (30) NOT NULL,
    location VARCHAR (20) NOT NULL
)
GO
INSERT INTO tsps VALUES
(1, 'BITL', 'Dhanmondi'),
(2, 'BITLM', 'Mirpur'),
(3, 'DIIT', 'Dhanmondi'),
(4, 'CRVL', 'Elephant Road')
GO
INSERT INTO trainees VALUES
```

```
(1, 'Shahid', 'Mohammedpur'),
(2, 'Iqbal', 'Mirpur'),
(3, 'Azmal', 'Dhanmondi'),
(4, 'Akib', 'Dhanmondi')
GO
SELECT * FROM trainees
GO
SELECT * FROM tsps
GO
```

- Data in trainees tables

id	name	residence
1	BITL	Dhanmondi
2	BITLM	Mirpur
3	DIIT	Dhanmondi
4	CRVL	Elephant Road

- Data in tsps. Table

id	name	location
1	Shahid	Mohammedpur
2	Iqbal	Mirpur
3	Azmal	Dhanmondi
4	Akib	Dhanmondi

- Suppose we want to get list trainees with residence, along with tsp name in his/her residence area. In addition we want view location of tsps where no trainee resides.
- In this situation FULL OUTER JOIN is required

Try this

```
SELECT tr.name 'trainee', residence,
ts.name 'tsp', location
FROM trainees tr
FULL OUTER JOIN tsps ts
ON tr.residence = ts.location
GO
```

- Result

trainee	residence	tsp	location
Shahid	Mohammedpur	NULL	NULL
Iqbal	Mirpur	BITLM	Mirpur
Azmal	Dhanmondi	BITL	Dhanmondi
Azmal	Dhanmondi	DIIT	Dhanmondi
Akib	Dhanmondi	BITL	Dhanmondi
Akib	Dhanmondi	DIIT	Dhanmondi
NULL	NULL	CRVL	Elephant Road

Cross JOIN

- Cross join is a strange type of join, it has no ON operation
- A cross join that does not have a WHERE clause produces the Cartesian product of the tables involved in the join.
- The size of a Cartesian product result set is the number of rows in the first table multiplied by the number of rows in the second table.

Try this

```
SELECT tr.*, ts.*
FROM trainees tr
CROSS JOIN tsps ts
GO
```

- Result

id	name	residence	id	name	location
1	Shahid	Mohammedpur	1	BITL	Dhanmondi
2	Iqbal	Mirpur	1	BITL	Dhanmondi
3	Azmal	Dhanmondi	1	BITL	Dhanmondi
4	Akib	Dhanmondi	1	BITL	Dhanmondi
1	Shahid	Mohammedpur	2	BITLM	Mirpur
2	Iqbal	Mirpur	2	BITLM	Mirpur
3	Azmal	Dhanmondi	2	BITLM	Mirpur
4	Akib	Dhanmondi	2	BITLM	Mirpur
1	Shahid	Mohammedpur	3	DIIT	Dhanmondi
2	Iqbal	Mirpur	3	DIIT	Dhanmondi
3	Azmal	Dhanmondi	3	DIIT	Dhanmondi
4	Akib	Dhanmondi	3	DIIT	Dhanmondi
1	Shahid	Mohammedpur	4	CRVL	Elephant Road
2	Iqbal	Mirpur	4	CRVL	Elephant Road
3	Azmal	Dhanmondi	4	CRVL	Elephant Road
4	Akib	Dhanmondi	4	CRVL	Elephant Road